

МИНИСТЕРСТВО ТРАНСПОРТА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ АГЕНТСТВО ЖЕЛЕЗНОДОРОЖНОГО ТРАНСПОРТА
Государственное бюджетное образовательное учреждение
высшего образования
**«ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПУТЕЙ СООБЩЕНИЯ ИМПЕРАТОРА АЛЕКСАНДРА I»**

Кафедра «ИНФОРМАЦИОННЫЕ И ВЫЧИСЛИТЕЛЬНЫЕ СИСТЕМЫ»

Дисциплина: «Программирование Ассемблер»

О Т Ч Е Т
по лабораторной работе № 3.2

Выполнил студент
Факультета АИТ
Группы ИВБ-515
Принял

Нартов С. А.

Кукин М. Ю.

Санкт-Петербург
2026

Задача

Написать программу, в которой: `сR`

- осуществляется поэлементная генерацию неупорядоченного массива из 20 случайных чисел (максимальное значение случайного числа = номер по списку * 20) – подробнее о генерации в описании работы Лабораторной работы № 3 часть 1.
- вывод данного массива в одну строку (элементы должны быть разделены минимум одним пробелом);
- сортировка массива в соответствии со схемой алгоритма;
- вывод отсортированного массива в одну строку (элементы должны быть разделены минимум одним пробелом).

Листинг

```

.586p
.model flat, stdcall
option casemap: none

include C:\masm32\include\windows.inc
include C:\masm32\include\kernel32.inc
includelib C:\masm32\lib\kernel32.lib
include C:\masm32\include\user32.inc
includelib C:\masm32\lib\user32.lib
include C:\masm32\include\masm32.inc
includelib C:\masm32\lib\masm32.lib
include C:\masm32\include\msvcrt.inc
includelib C:\masm32\lib\msvcrt.lib
include C:\masm32\macros\macros.asm

.data
    arr            dd 20 dup(?)          ; array of 20 elements
    arrSize        dd 20                ; array size
    maxValue       dd 420               ; maximum value
    buffer         db 12 dup(?)         ; buffer for number output
    rand_buf       db 5 dup(?)          ; buffer for random number generation

    msg_original   db "Original array:", 13, 10, 0
    msg_sorted     db 13, 10, "Sorted array:", 13, 10, 0
    msg_epoch      db 13, 10, "Epoch %d", 13, 10, 0
    msg_format_num db "%d ", 0
    msg_format_last db "%d", 13, 10, 0
    msg_newline    db 13, 10, 0
    msg_press_key  db "Press any key for exit...", 0

.code
print_array PROC uses ecx esi
    mov ecx, 20                ; number of elements
    mov esi, offset arr        ; pointer to array

print_loop_func:
    push ecx

    mov eax, [esi]             ; current element
    cmp ecx, 1
    jg print_space_func
    invoke crt_printf, addr msg_format_last, eax
    jmp print_next_func

print_space_func:
    invoke crt_printf, addr msg_format_num, eax

print_next_func:
    add esi, 4
    pop ecx

```

```

    loop print_loop_func

    ret
print_array ENDP

shaker_sort PROC uses eax ebx ecx edx esi edi
    mov esi, 0                ; left = 0
    mov edi, 19               ; right = n-1 (19 for 20 elements)
    mov ebx, 1                ; swapped flag = 1
    mov dword ptr [buffer], 1 ; epoch counter

sort_loop:
    cmp ebx, 0                ; if swapped == 0, sorting is done
    je sort_done

    mov ebx, 0                ; swapped = 0

    mov ecx, esi              ; i = left

left_to_right:
    cmp ecx, edi              ; i < right ?
    jge right_to_left_start

    mov eax, [offset arr + ecx*4] ; arr[i]
    mov edx, [offset arr + ecx*4 + 4] ; arr[i+1]

    cmp eax, edx              ; arr[i] < arr[i+1] ? (for descending order)
    jge no_swap_right

    ; Swap arr[i] and arr[i+1]
    mov [offset arr + ecx*4], edx
    mov [offset arr + ecx*4 + 4], eax
    mov ebx, 1                ; swapped = 1

no_swap_right:
    inc ecx
    jmp left_to_right

right_to_left_start:
    cmp ebx, 0                ; if no swaps were made, exit
    je sort_done

    dec edi                  ; right--

    mov ecx, edi              ; i = right

right_to_left:
    cmp ecx, esi              ; i > left ?
    jle epoch_display

    mov eax, [offset arr + ecx*4] ; arr[i]

```

```

    mov edx, [offset arr + ecx*4 - 4] ; arr[i-1]

    cmp edx, eax                ; arr[i-1] < arr[i] ? (for descending order)
    jge no_swap_left

    ; Swap arr[i-1] and arr[i]
    mov [offset arr + ecx*4], edx
    mov [offset arr + ecx*4 - 4], eax
    mov ebx, 1                  ; swapped = 1

no_swap_left:
    dec ecx
    jmp right_to_left

epoch_display:
    inc esi                    ; left++

    ; Display current array state (epoch)
    push edi
    push esi
    push ebx

    invoke crt_printf, addr msg_epoch, dword ptr [buffer]
    call print_array

    inc dword ptr [buffer]      ; increment epoch counter

    pop ebx
    pop esi
    pop edi

    jmp sort_loop

sort_done:
    ret
shaker_sort ENDP

main PROC
    ; Generate random array
    mov ecx, 20                ; array element counter
    mov esi, offset arr        ; pointer to array

gen_loop:
    push ecx
    push esi

    ; Generate random msging of 4 characters
    lea edi, rand_buf
    mov ecx, 4

gen_char:

```

```

push ecx

; Random number generation (RDRAND)
@@:
db 0Fh, 0C7h, 0F0h
jnc @B

xor edx, edx
mov ecx, 94
div ecx
add edx, 33

mov al, dl
stosb

pop ecx
loop gen_char

mov byte ptr [edi], 0

invoke atodw, addr rand_buf

xor edx, edx
mov ecx, maxValue
inc ecx
div ecx
mov eax, edx

pop esi
mov [esi], eax
add esi, 4

pop ecx
loop gen_loop

;prn arr
invoke crt_printf, addr msg_original
call print_array

call shaker_sort

; Display sorted array
invoke crt_printf, addr msg_sorted
call print_array
invoke crt_printf, addr msg_newline

inkey "Press any key for exit..."
invoke ExitProcess, 0

```

```

main ENDP
end main

```

